pfi

Python for Fixed Income

COURSE 7 · NLP, RAG, AND LLMS

Text in Python: FOMC Statements and an Indenture

WEEK 1 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Text as data, in three weeks

Week 1: Text as data

Strings, TF-IDF, a classifier, word vectors

Week 2: Transformers on the CPU

Embeddings, search by meaning, EDGAR

Week 3: LLMs, with checks

Covenant extraction checked against the text

Today's two documents

226 FOMC statements

2000-02-02 to 2026-09-16, the Board's own text

One indenture, from EDGAR

Boyd Gaming Corporation, 4.750% Senior Notes due 2031: the teaching indenture for all three weeks

Today: text as Python sees it. No model, no key.

Real documents, described

An FOMC statement's output

Allowed names	Never called
count of words from the course's list	hawkish or dovish
dictionary score	a signal or a policy view
the action the statement states	a forecast of rates

An indenture's terms

Allowed names	Never called
the indenture states	loose, tight, weak, strong
extracted value, with its quote	covenant-lite or protective
passed the source check	a credit view or recommendation

Text is a string

1,254

characters in the first statement

6

text.count("rate")

5

the whole word "rate"

```
text[:300]                  # a slice: the first 300 characters
"Inflation" in text         # False: in is exact about capitals
words = text.split()        # pieces, with punctuation glued on
text.count("rate")          # 6: also inside 'generate'
```

A substring count is not a word count, in Python or in Excel.

226 statements, 213 after scheduled meetings and 13 unscheduled. Source: the Board of Governors, federalreserve.gov, one page per statement.

Why the file starts in 2000

February 1994	The FOMC first announced the outcome of a meeting.
February 1995	All changes in the stance of policy communicated immediately.
January 2000	A statement after each regularly scheduled meeting.

The Board's FAQ dates statements after every meeting from May 1999. The course quotes both.

Sources, read 8 October 2026: federalreserve.gov/monetarypolicy/fomc_historical.htm (Transcripts and other historical materials); federalreserve.gov/faqs/federal-open-market-committee-fomc-not-public.htm

How normalizing works

The problem	Two spellings that look alike are different text until something makes them the same
Step 1	Unicode NFC writes every composed character in one standard form
Step 2	A character map swaps each look-alike for its plain keyboard character, in one pass
Step 3	Every run of white space becomes one space, or one newline where it held a line break
What comes out	A string of plain characters; you control the map, a dictionary you can read and extend

Characters that look alike

Code	Name	Times	Statements
U+2011	NON-BREAKING HYPHEN	50	34
U+2013	EN DASH	3	3
U+2014	EM DASH	2	1
U+2019	RIGHT SINGLE QUOTATION MARK	25	19

Searching "longer-term"

Plain hyphen: 102 statements. With U+2011: 7 more, missed. After normalize_text: 109.

```
textkit.normalize_text(t)    # NFC, the map, white space tidied: 0 characters outside ASCII left
```

The file keeps the Board's characters. Normalizing happens in code, on both sides.

How a regular expression works

The problem	The same fact is worded a little differently each year, so fixed text finds some statements and misses others
The pattern	Describes the shape: \d+ digits, a set in brackets, (...) a group, | alternatives
The search	re.search returns the first place the pattern fits, with its groups; re.findall returns every place
You control	The pattern and its flags, such as re.IGNORECASE
What comes out	A match object, or None when the text does not fit: test every pattern on every document

Regular expressions: the target range

```
NUMBER = r"(?:\d+[- ]\d+/\d+|\d+/\d+|\d+)"        # 5-3/4, 1 3/4, 1/4, 6
RANGE  = ...federal funds rate ... (?P<low>NUMBER) to (?P<high>NUMBER) percent
TARGET = ...target for the federal funds rate ... (?P<target>NUMBER) percent
```

144

state a range, from 2008-12-16

76

a single target, to 2008-10-29

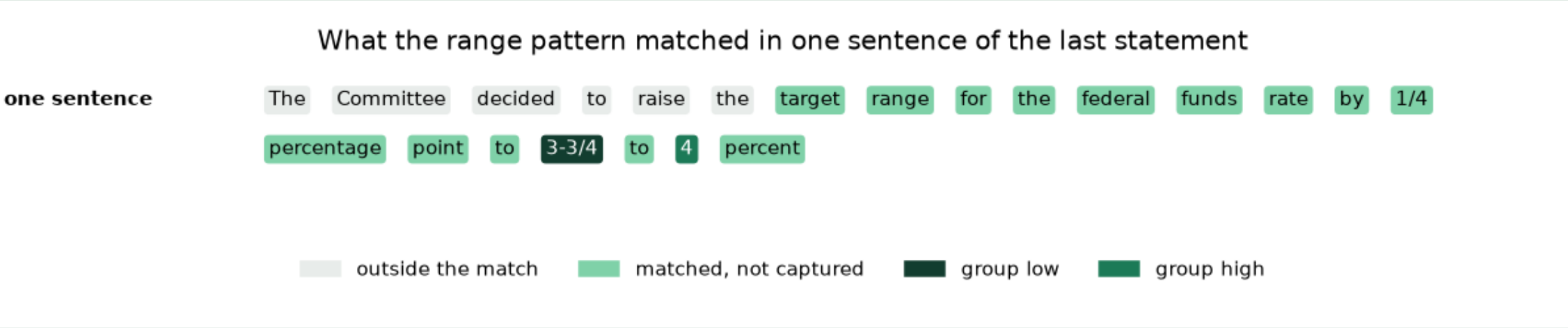
6

neither: all unscheduled

On the raw text: 132 ranges, 12 missed, every one written with U+2011.

A pattern is a claim about how a document is written. The counts test it.

What the pattern matched



3-3/4

group low, as text

4

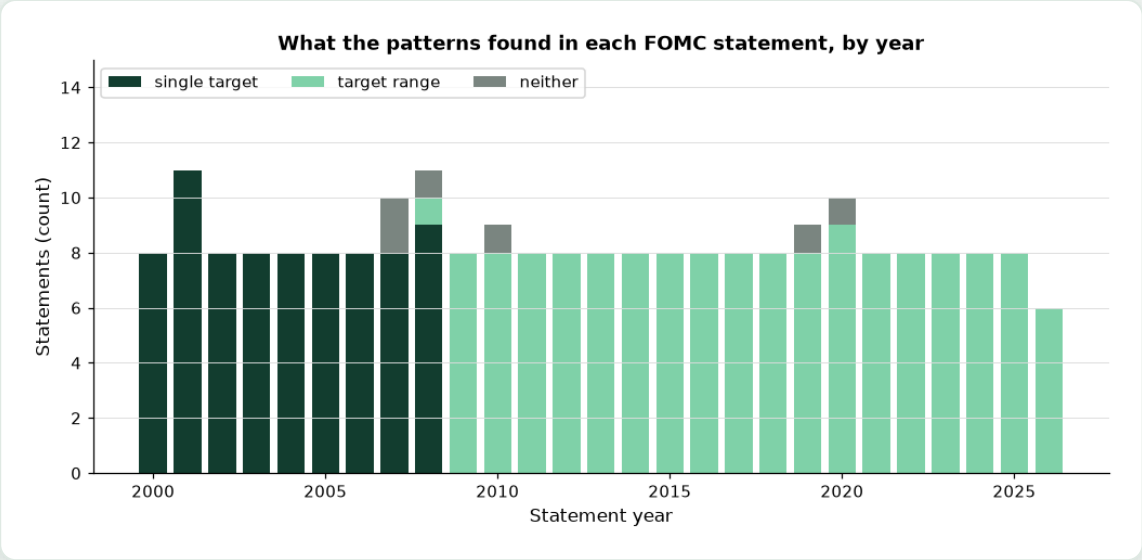
group high, as text

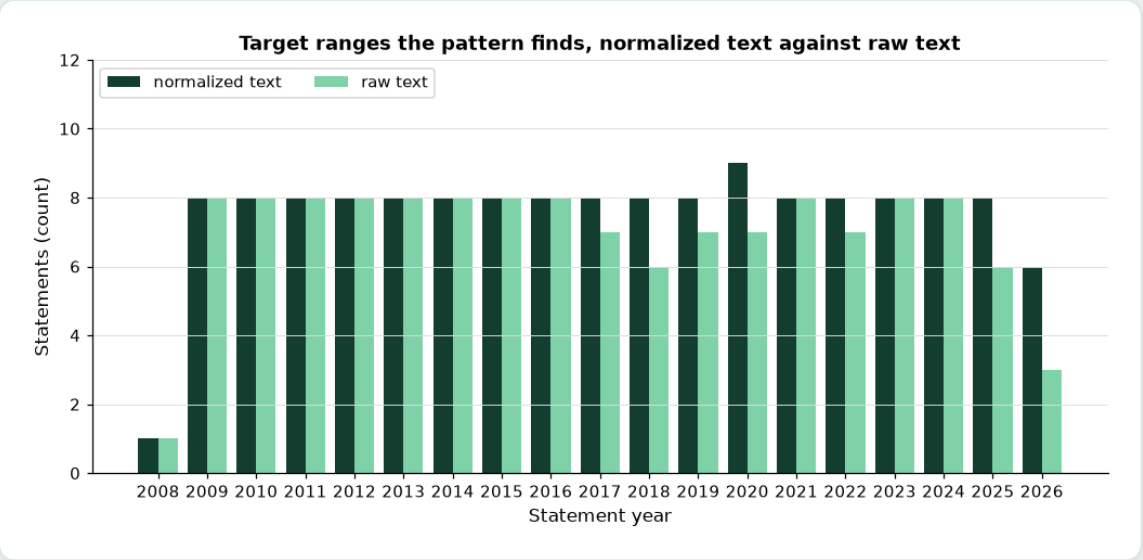
16 of 22

pieces the match covers

The pattern knows the sentence's shape. What the numbers mean is the next step.

What the patterns found, by year





From the styled table: ranges 144 on the normalized text, 132 on the raw; "none" 6 and 18.

Drawn by year, a silent failure has a shape you can see.

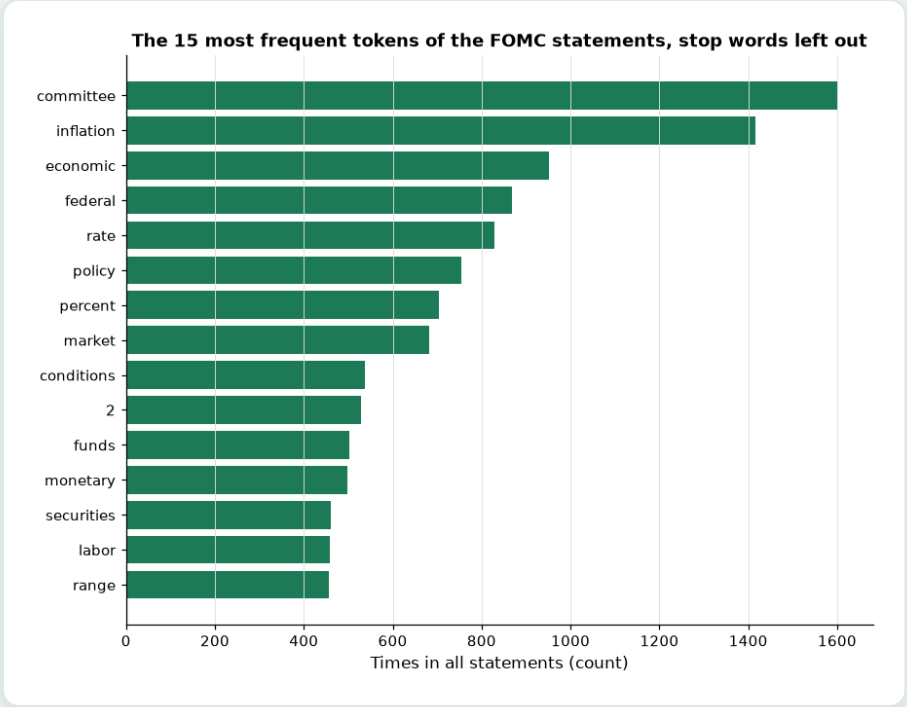
How a tokenizer works

The problem	A word is counted the same way only if it is cut out of the text the same way every time
The rule	Lower-case the normalized text, then take every run of letters and digits
Inside a run	A single hyphen, apostrophe, or full stop may join the parts; other punctuation is dropped
You control	The pattern, whether to lower-case, and which stop words to leave out
What comes out	A list of tokens in the text's order: the unit every count, column, and classifier uses

Tokens

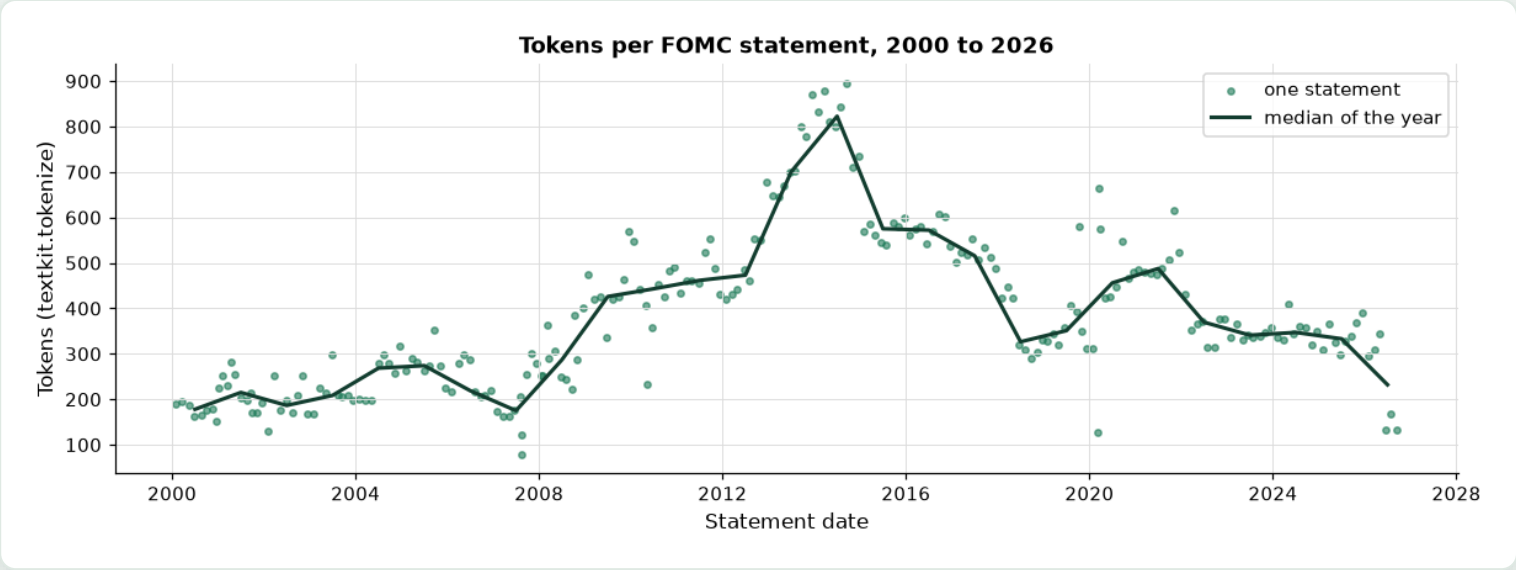
```
TOKEN_PATTERN = r"[a-z0-9]+(?:[-'.][a-z0-9]+)*"
tokenize("agency mortgage-backed securities")
  ['agency', 'mortgage-backed', 'securities']
tokenize("0 to 1/4 percent")
  ['0', 'to', '1', '4', 'percent']
```

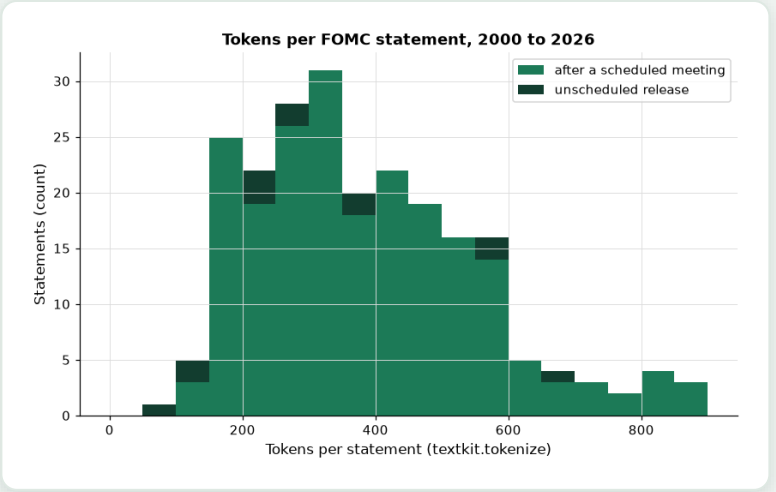
First statement: 190 tokens against 188 split pieces. All statements: 86,576 tokens, 1,910 distinct; stop words are a share of 0.3776.



Tokens are for counting words. Patterns are for reading numbers.

The statements got longer





A count describes each document by date. It does not say why the length changed.

How find_sections works

The problem	An indenture is a long text, and a lawyer cites it by number: Section 4.12
The pattern	Start of a line, the word Section, a number, a full stop, a title starting with a capital
Twice over	The contents holds the same headings: each number matches twice, so keep the last match, the body's
Each section	Ends where the next kept one starts
What comes out	One row per section: number as text, title, start and end characters

A first look at an indenture

63,595

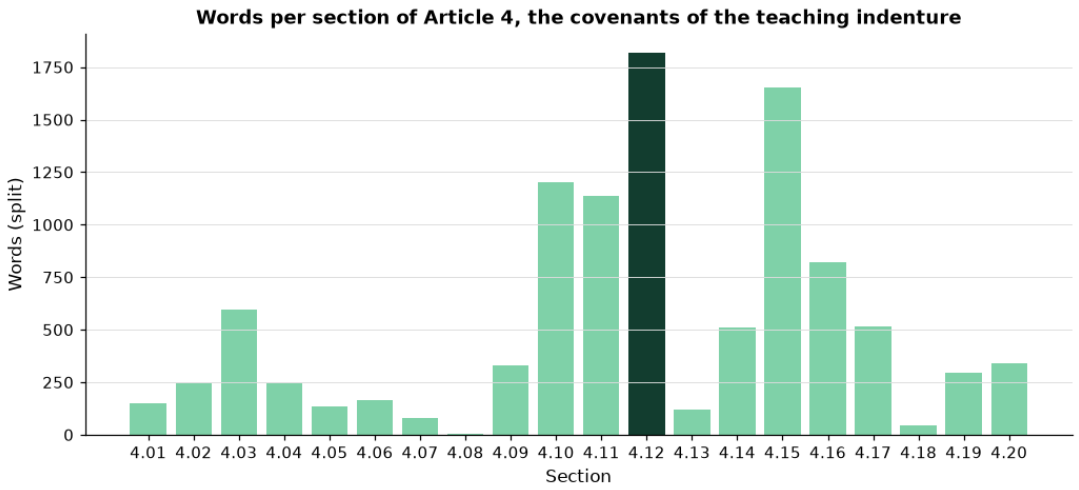
words in the indenture

214

heading lines for 107 sections

198,454

where the body's 4.12 starts



Why the contents go

Every number matches twice. find_sections keeps the last heading: the body's.

Source: Boyd Gaming Corporation, indenture for the 4.750% Senior Notes due 2031, Exhibit 4.1 to the 8-K filed 2021-06-08, accession 0001193125-21-185495.

How defined terms are found

The problem	Every capitalized word means exactly what Section 1.01 says, wherever it is used
The pattern	A line that starts with a quoted term, then means or has the meaning
It keeps	The term, the kind of phrase after it, and where the definition starts; quotes made straight first
You control	The pattern and the stretch of text searched
What comes out	One row per definition found, only as complete as the pattern: check what it misses

Defined terms, and the ones the pattern misses

136

"Term" means, or has the meaning

148

lines that start with a quoted term

12

written another way: missed

How the missed definitions are written

"... shall mean ...", "... shall be deemed to occur if ...", and "Consolidated Total Assets" of any Person as of any date means ...

Test every pattern on the document it runs on. Day 12 widens this one.

Source: Boyd Gaming Corporation, indenture for the 4.750% Senior Notes due 2031, Exhibit 4.1 to the 8-K filed 2021-06-08, accession 0001193125-21-185495, Section 1.01.

How a stemmer works

The problem	Related spellings of a word count as different words
The rules	A stemmer cuts suffixes from a list: restricted, restricts, restriction all become restrict
The catch	The rules do not read the sentence, so words that differ by context are merged
You control	Which stemmer to use, if any; the course uses none
What comes out	A smaller vocabulary and pooled counts: the bars show what pooling hides

Stemming, named and not used

Token	Count
Restricted	412
restrictions	43
restriction	7
RESTRICTED	6
restrict	3

The capital letter carries meaning

Restricted Subsidiary 250 times, Restricted Payment 29: both defined terms. A stemmer would put them in one count of "restrict" with the ordinary word.

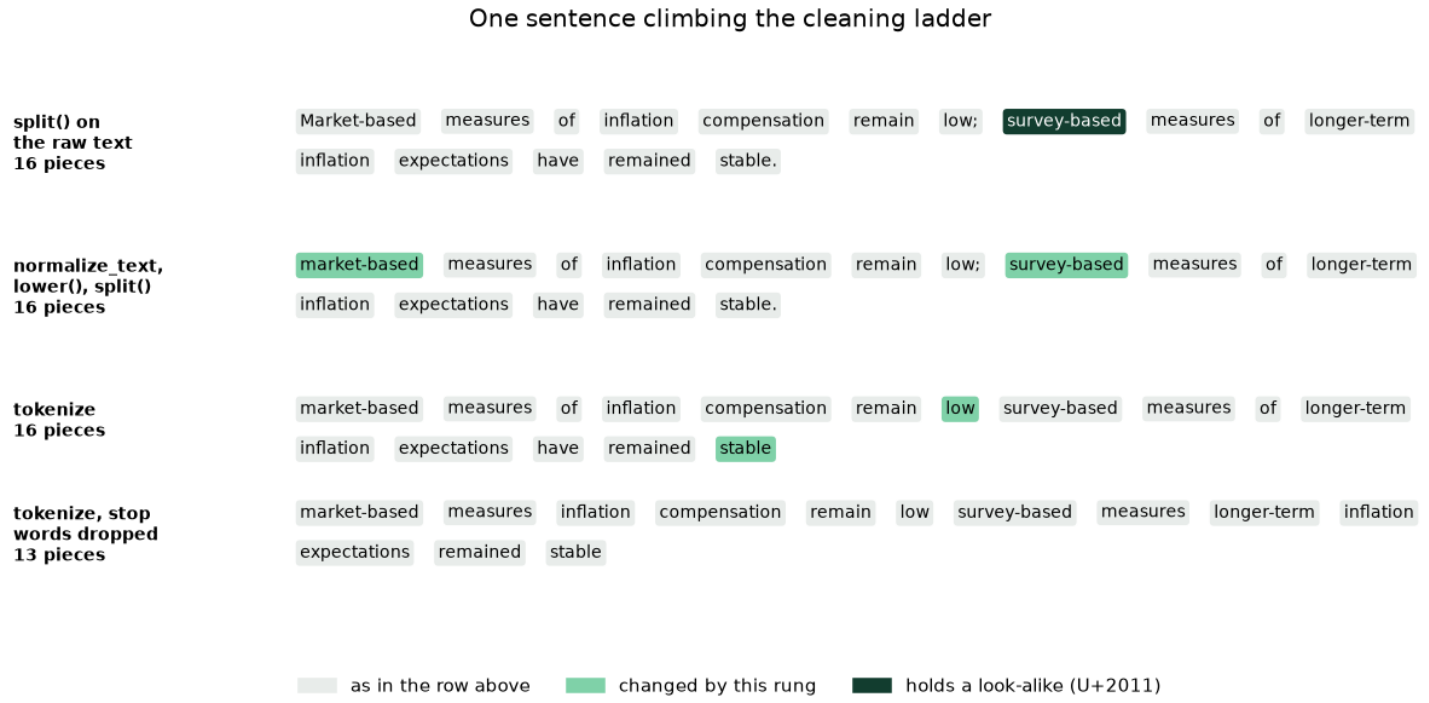
The course keeps words whole. No stemming library is used.

Source: Boyd Gaming Corporation, indenture for the 4.750% Senior Notes due 2031, Exhibit 4.1 to the 8-K filed 2021-06-08, accession 0001193125-21-185495.

How the cleaning ladder works

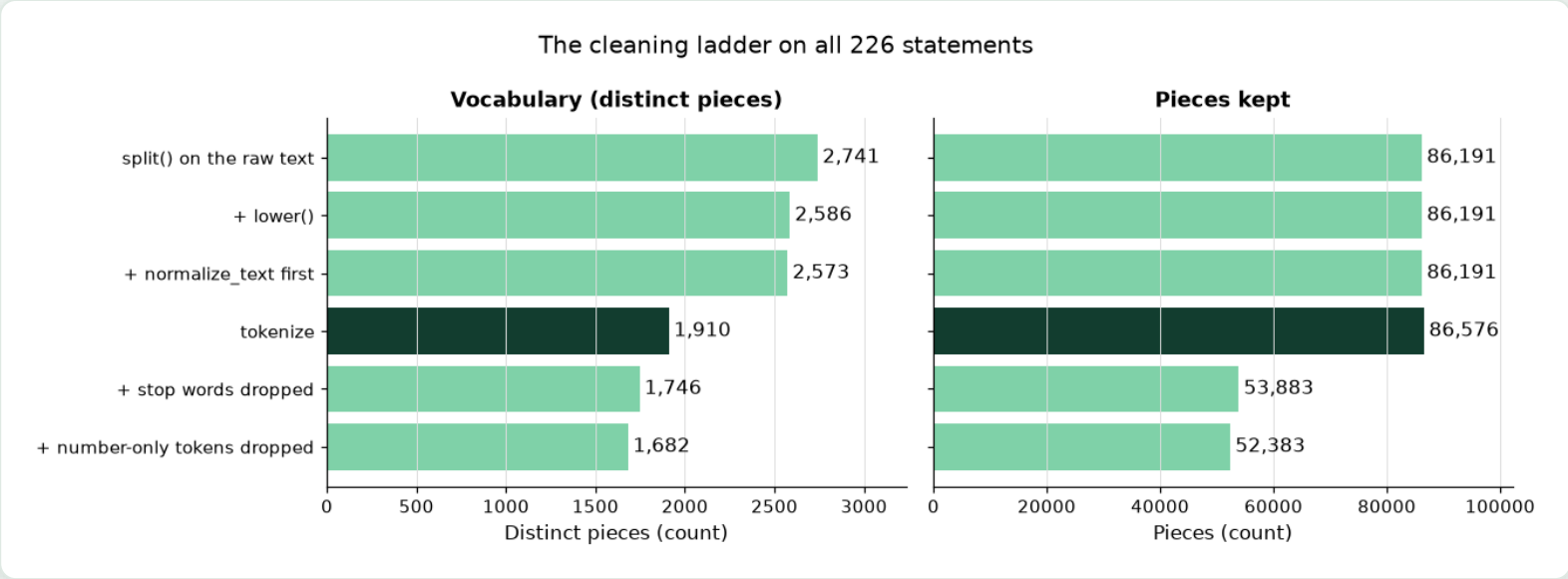
The problem	Each cleaning rule changes what a count means, and which rungs to climb is a choice
The rungs	Lower-case; normalize; tokenize; drop stop words; drop number-only tokens
Each rung	Removes some pieces and merges some spellings, so the vocabulary shrinks as you climb
You control	Which rungs to climb; the same text gives a different table at each
What comes out	A list of pieces to count; the figures show what each rung did

One sentence on the ladder



Pieces at the six rungs: 16, 16, 16, 16, 13, 13. The dark box is the non-breaking hyphen; light green is what a rung changed.

The ladder on every statement



Rung	Distinct	Removed
raw split	2,741	0
+ lower	2,586	155
+ normalize	2,573	13
tokenize	1,910	663
+ stop words	1,746	164
+ numbers	1,682	64

Tokenize removes the most distinct pieces; stop words remove the most tokens. The course takes the tokenize rung for every count and keeps the numbers.

Excel side by side

Excel side by side (tested in Excel)

`=LEN(A2)`	1,254, equal to len(text)
`=SEARCH("inflation",A4) and =FIND("inflation",A4)`	195 and 336: Python + 1
`=(LEN(A2)-LEN(SUBSTITUTE(LOWER(A2),"inflation","")))/LEN("inflation")`	2; tokenize: 1
`=TEXTBEFORE(TEXTAFTER(A3,"percentage point to ")," percent")`	3-3/4 to 4
`=ROWS(TEXTSPLIT(A3,," "))`	30; tokens 32
`=REGEXEXTRACT(A3,"[0-9/-]+ to [0-9/-]+ percent")`	3-3/4 to 4 percent
`=REGEXEXTRACT(A4,"[0-9/-]+ to [0-9/-]+ percent")`	#N/A on U+2011
`=REGEXEXTRACT(SUBSTITUTE(A4,UNICHAR(8209),"-"),"[0-9/-]+ to [0-9/-]+ percent")`	3-1/2 to 3-3/4 percent
`=WORDCOUNT(A2)`	#NAME?

No tokenizer in Excel. A cell keeps 32,767 characters: one paragraph per row (1,714 rows).

Save the day: start textkit

1. Init

The work folder, local settings, the guard

2. .gitignore

adds .env and cache/ from day 1

3. Commit

mlkit beside textkit and its tests

4. Status

the data copies stay untracked

```
 .gitignore      |   5 +
 mlkit.py        | 568 ++++++
```

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Units, conventions, and what the function must never do: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: the splitter never normalized U+2011: "longer" 393 times, plus 7 odd one-word "longer-term".

Code that runs is not the same as code that is right.

What you can do now

- Treat a document as a string; a substring is not a word
- Make look-alike characters plain with normalize_text
- Pull the target range out of every statement, checked
- Find an indenture's sections and its defined terms
- Do the same string work in Excel, with the same traps

Week 1, Day 2 Bag of words and TF-IDF